

Таблица определения приоритета: API

Предисловие:

Стоит учитывать, что от задачи к задаче её ключевая особенность будет отличаться и если суть вашей задачи в "Поиск по ID", то ваш Блокирующий уровень - это отсутствие результата поиска по существующему ID, а если ваша задача "публикация и обновление данных", то проверяем и обязательные параметры, маскирование если имеется и результат публикации данных в базе данных. Важно! Если ручка реализованная по задаче предоставляет данные для UI это тоже критическая часть проверки этой ручки и доп. проверка. При передаче в атвоматизацию учитывай сквозную цепочку успешных тестов или шагов теста.

Таблица определения приоритета: API

Тут мы смотрим на то, как передаются данные «под капотом»:

Уровень	Критерий	Описание влияния
Блокирующий	Главный - успешный путь	Самая важная команда. Если она не работает, функция полностью бесполезна.
Критический	Безопасность и главная логика	Работают ли права доступа и правильно ли сохраняются ключевые данные.
Значительный	Проверка входных данных	Как система реагирует на пустые поля, неверный формат текста или чисел.
Мелкий	Скорость и детали	Долго ли система думает и все ли дополнительные параметры на месте.
Низкий	Тексты и оформление	Опечатки в системных сообщениях или мелкие неточности в описании.

Если есть сложности с определением критичности, попробуй ответить на базовые вопросы:

1. Это единственный успешный путь фичи? (Без этого теста вообще нельзя сказать, что задача сделана)
> Если да = **Блокирующий**
2. Поломка здесь приведет к потере данных или краже доступа? (Пользователь увидит чужое или данные пропадут, не сохранятся)
> Если да = **Критический**
3. Сломан ли основной механизм работы с данными? (Например, для API: типы полей; для UI: неверные цифры на экране)
> Если да = **Критический**
4. Это мешает пользоваться функцией, но есть другой способ? (Например: не работает фильтр или поиск, но список виден)
> Если да = **Значительный**
5. Это касается только внешнего вида или удобства? (Отступы, шрифты, цвета, мелкие задержки в ответах)
> Если да = **Мелкий**

6. Это просто опечатка или неточность в тексте? (Не влияет на смысл и работу программы)

> Если да = **Низкий**

На примерах:

1. Тест «Создание новой ветки»:

Это единственный успешный путь? Да.

Итог: **Блокирующий** (Пойдет в автоматизацию первым).

2. Тест «Ошибка при вводе спецсимвола в название ветки» (API):

Это основной успех? Нет.

Будет кража данных? Нет.

Это ломает механизм данных? Нет, это просто обработка ошибки.

Итог: **Значительный**.

3. Тест «Неверный отступ у кнопки удаления ветки» (UI):

Мешает пользоваться? Нет.

Это внешний вид? Да.

Итог: **Мелкий**.

Если вы видите, что в «Блокирующие» и «Критические» попадает слишком много тестов, пересмотрите вопрос №3 — действительно ли эта ошибка «убьет» данные, или это просто неудобство?

Примечание:

Как оставить только 20% важных тестов уровня Критический и Блокирующий:

1. В Блокирующие пишем только один тест — «Всё ввели верно и всё сработало».
2. В Критические добавляем только проверку прав (можно ли войти) и точность главных данных.
3. Всё остальное (проверки на ошибки, пустые поля, кривую верстку и опечатки) смело отправляем в уровни Значительный и ниже.